ÉPREUVE E6

Chaîne CI/CD conteneurisée

Module	Projet Intégration et Déploiement Continus
Durée	3 heures 30
Compétences évaluées	C21, C22, C23, C24, C25
Documents autorisés	Tous documents et accès internet
Matériel	PC avec Docker et Git installés + accès internet
Modalité	Individuel – Anonyme

⚠️ CONSIGNES DE RENDU

Format du rendu : Un **unique fichier PDF** complet contenant toutes vos réponses, schémas, captures d'écran, fichiers de configuration et extraits de code. Le document doit être professionnel et inclure obligatoirement :

- **Une page de garde** (titre de l'épreuve, module, date, mention « Anonyme »)
- **Un sommaire**
- **Des sections clairement identifiées** correspondant à chaque partie et question du sujet
- **Les captures d'écran, schémas et code** intégrés directement dans le PDF (pas de fichiers annexes)
- **Pour les tiers-temps** : Parties en vert facultatives.

Justification systématique : Chaque choix technique, même évident, doit être justifié par au moins une phrase. Un choix non justifié sera pénalisé, même s'il est correct.

Dépôt du rendu : Le PDF doit être déposé **exclusivement sur Moodle**, dans l'espace prévu à cet effet, **avant la fin de l'épreuve**. Aucun rendu par e-mail ou Teams ne sera accepté.

Anonymat : Le rendu est **anonyme**. Ne faites figurer ni votre nom, ni votre prénom, ni votre numéro étudiant sur le document. Moodle associe automatiquement votre identité à votre dépôt.

*Le dépôt Git (GitHub/GitLab) utilisé pendant l'épreuve doit être rendu **public** ou accessible via un lien partagé inclus dans votre PDF. Il ne sera consulté qu'en complément du PDF.*

🤖 UTILISATION DE L'IA GÉNÉRATIVE

L'utilisation d'outils d'IA générative (ChatGPT, Claude, Copilot, Gemini, etc.) est **autorisée** pendant cette épreuve. Cependant, elle doit être **systématiquement transparente et justifiée**. La transparence sur l'usage de l'IA fait partie intégrante de la notation.

Règles à respecter impérativement :

- Si vous utilisez l'IA pour vous aider (débuguer, générer du code, rappeler un concept, rédiger un paragraphe, etc.), vous devez le **signaler explicitement** dans votre rapport à l'endroit concerné.
- Pour chaque utilisation, précisez : le **fournisseur et le modèle** utilisé (ex : ChatGPT GPT-4o, Claude Sonnet, GitHub Copilot…), le **prompt exact** que vous avez soumis, et la **réponse obtenue** (ou un résumé si elle est longue).
- Expliquez **pourquoi** vous avez choisi d'utiliser l'IA plutôt qu'une recherche classique (documentation, Stack Overflow, cours). Exemples de justifications acceptées : gain de temps sur un problème bloquant, reformulation d'un concept mal compris, génération d'un squelette de code à adapter, etc.
- Indiquez si vous avez **modifié ou adapté** la réponse de l'IA, et si oui, en quoi et pourquoi.

Ce qui sera valorisé :

- Une utilisation pertinente et ciblée de l'IA (pas du copier-coller intégral sans recul)
- La transparence totale : mieux vaut déclarer une utilisation que de la masquer
- La capacité à expliquer et critiquer la réponse de l'IA (montrer que vous comprenez)

Ce qui sera pénalisé :

- L'utilisation non déclarée d'IA détectée lors de la correction
- Le copier-coller brut d'une réponse IA sans adaptation, sans explication et sans recul critique
- L'incapacité à expliquer ou défendre un élément de votre rendu produit par IA

Conseil : vous pouvez dédier une sous-section « Utilisation de l'IA » à la fin de chaque partie, ou annoter directement chaque passage concerné. L'important est que le correcteur puisse identifier clairement ce qui vient de vous et ce qui vient d'un outil.

Barème

Partie	Points	Durée indicative
Partie 1 – Git et gestion de versions	20 pts	~40 min
Partie 2 – Conteneurisation Docker	25 pts	~50 min
Partie 3 – Pipeline CI/CD	30 pts	~60 min
Partie 4 – Orchestration et supervision	15 pts	~30 min
Partie 5 – Documentation	10 pts	~10 min
TOTAL	**100 pts**	**3h30**

Contexte général

Vous êtes développeur DevOps chez **VitalSync**, une startup développant une application de suivi médical et sportif. Votre mission est de mettre en place la chaîne d'intégration et de déploiement continus pour l'application.

L'application à conteneuriser comprend :

- **Un back-end API** – serveur Node.js/Express exposant une API REST sur le port 3000
- **Un front-end** – application React servie par Nginx sur le port 80
- **Une base de données** – PostgreSQL (image officielle)

Vous utiliserez des **applications squelettes** minimalistes fournies ci-dessous. L'objectif n'est pas de développer l'application mais de construire toute la chaîne DevOps autour.

Fichiers de départ

Créez un répertoire **vitalsync/** avec la structure suivante :

```
vitalsync/
├── backend/
│   ├── server.js
│   ├── package.json
│   └── test/
│       └── health.test.js
├── frontend/
│   ├── index.html
│   └── nginx.conf          (à créer)
└── docker-compose.yml      (à créer)
```

backend/server.js

```
const express = require("express");
const app = express();

app.get("/health", (req, res) => {
```

```
  res.json({ status: "ok", timestamp: new Date() });
});

app.get("/api/activities", (req, res) => {
  res.json([]);
});

app.listen(3000, () => console.log("VitalSync API on :3000"));
```

backend/package.json

```
{
  "name": "vitalsync-api",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "test": "jest"
  },
  "dependencies": { "express": "^4.18.0" },
  "devDependencies": {
    "jest": "^29.0.0",
    "supertest": "^6.0.0"
  }
}
```

backend/test/health.test.js

```
const request = require("supertest");
const express = require("express");
const app = express();
app.get("/health", (req, res) =>
  res.json({ status: "ok" })
)

test("GET /health returns 200", async () => {
  const res = await request(app).get("/health");
  expect(res.statusCode).toBe(200);
  expect(res.body.status).toBe("ok");
});
```

frontend/index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>VitalSync</title>
</head>
<body>
  <h1>VitalSync - Suivi médical et sportif</h1>
  <p id="status">Chargement...</p>
  <script>
    fetch("/api/health")
      .then(r => r.json())
      .then(d => {
        document.getElementById("status")
```

```
            .textContent = "API: " + d.status;
        });
    </script>
  </body>
  </html>
```

Partie 1 – Git et gestion de versions (20 points)

Exercice 1 – Initialisation et structuration du dépôt (8 points)

a) Créez un dépôt Git (GitHub ou GitLab) pour le projet VitalSync. Initialisez-le avec les fichiers de départ fournis.

b) Mettez en place une stratégie de branches Gitflow. Créez au minimum les branches : main, develop, et une branche feature (ex : feature/docker-setup).

c) Créez un fichier .gitignore adapté au projet (Node.js, fichiers d'environnement, etc.).

d) Configurez une règle de protection sur la branche main : interdiction du push direct, obligation de passer par une Pull/Merge Request.

📌 **Attendu dans le PDF :**

- Screenshot de la page du dépôt Git montrant l'URL du repo
- Screenshot de la liste des branches créées (interface web ou terminal git branch -a)
- Contenu complet du fichier .gitignore collé dans le PDF
- Justification de chaque entrée du .gitignore (même une phrase courte, ex : « node_modules/ car les dépendances ne doivent pas être versionnées »)
- Screenshot des règles de protection de branche configurées
- Justification du choix GitHub ou GitLab (une phrase suffit)

Exercice 2 – Workflow Git et résolution de conflits (12 points)

a) Depuis develop, créez deux branches parallèles : feature/add-endpoint et feature/update-health. Sur chacune, modifiez server.js de manière à créer un conflit (les deux modifient la même zone du fichier).

b) Fusionnez la première branche dans develop. Tentez ensuite de fusionner la seconde et résolvez le conflit.

c) Expliquez par écrit la différence entre git merge et git rebase. Dans quel cas recommanderiez-vous l'un ou l'autre ?

d) Démontrez l'utilisation de conventions de commits (Conventional Commits). Appliquez ces conventions sur au moins 5 commits de votre dépôt.

📌 **Attendu dans le PDF :**

- Screenshot du diff ou du message de conflit Git dans le terminal
- Screenshot ou extrait du fichier en conflit montrant les marqueurs <<<<<<<, =======, >>>>>>>
- Screenshot du fichier après résolution du conflit
- Screenshot du commit de merge réussi (terminal ou interface web)
- Explication rédigée merge vs rebase (définitions + cas d'usage recommandé pour chacun)
- Screenshot de l'historique Git (git log --oneline ou interface web) montrant au moins 5 commits respectant Conventional Commits
- Justification du choix de la convention utilisée (une phrase)

Partie 2 – Conteneurisation Docker (25 points)

Exercice 3 – Dockerfile du back-end (10 points)

a) Rédigez un Dockerfile pour le back-end Node.js en utilisant un multi-stage build. Le premier stage installe les dépendances et exécute les tests. Le second produit une image légère de production (image alpine).

b) Ajoutez un commentaire en tête de votre Dockerfile expliquant pourquoi le multi-stage build est pertinent en termes de sécurité et de taille d'image.

c) Créez un fichier .dockerignore adapté et justifiez les entrées choisies.

📌 **Attendu dans le PDF :**

- Contenu intégral du Dockerfile collé dans le PDF (avec le commentaire explicatif en tête)
- Explication du multi-stage build : pourquoi 2 stages, quel est l'intérêt de chacun
- Justification du choix de l'image de base (ex : pourquoi node:20-alpine et pas node:20)
- Contenu du .dockerignore avec justification de chaque entrée
- Screenshot du build réussi (docker build) montrant les étapes
- Screenshot de docker images montrant la taille de l'image produite

Exercice 4 – Dockerfile du front-end (5 points)

a) Rédigez un Dockerfile pour le front-end servant index.html via Nginx. Créez également le fichier nginx.conf avec un proxy_pass vers le back-end pour les requêtes /api/*.

📌 **Attendu dans le PDF :**

- Contenu intégral du Dockerfile front-end
- Contenu intégral du fichier nginx.conf
- Explication du rôle du proxy_pass (pourquoi les requêtes /api/* sont redirigées vers le back-end)
- Justification du choix de l'image Nginx utilisée

Exercice 5 – Docker Compose (10 points)

a) Rédigez un fichier docker-compose.yml orchestrant les trois services : backend, frontend, database (PostgreSQL).

b) Configurez un réseau personnalisé (bridge) pour isoler les conteneurs. Expliquez l'intérêt de cette isolation.

c) Utilisez un volume persistant pour les données PostgreSQL. Expliquez ce qui se passerait sans ce volume.

d) Gérez les variables d'environnement sensibles via un fichier .env (non commité). Fournissez un fichier .env.example.

e) Vérifiez que docker-compose up lance les 3 services.

📌 **Attendu dans le PDF :**

- Contenu intégral du docker-compose.yml
- Explication du réseau personnalisé : pourquoi un bridge dédié, quel avantage par rapport au réseau par défaut
- Explication du volume persistant : qu'est-ce qui se passe si on fait docker-compose down sans volume
- Contenu du fichier .env.example (sans valeurs réelles, juste les clés avec des placeholders)
- Justification de chaque variable d'environnement déclarée
- Screenshot de docker-compose up montrant le démarrage des 3 services
- Screenshot de docker ps montrant les 3 conteneurs actifs avec leurs ports

Partie 3 – Pipeline CI/CD (30 points)

Exercice 6 – Configuration de la pipeline (20 points)

Mettez en place une pipeline CI/CD avec **GitHub Actions** ou **GitLab CI** (au choix). La pipeline doit comporter les étapes suivantes :

Étape 1 – Lint & Tests (6 pts)

- Installation des dépendances du back-end
- Exécution des tests unitaires (Jest)
- Ajout d'un linter ESLint avec une configuration basique

Étape 2 – Build Docker (6 pts)

- Construction des images Docker (backend + frontend)
- Tag des images avec le SHA du commit
- Push des images vers un registry (GHCR, Docker Hub ou GitLab Container Registry)

Étape 3 – Déploiement staging (8 pts)

- Déploiement automatique sur un environnement de test (peut être simulé en local via docker-compose)
- Vérification de santé (health check) du back-end après déploiement
- La pipeline doit échouer si le health check ne répond pas

📌 **Attendu dans le PDF :**

- Contenu intégral du fichier de pipeline (.github/workflows/*.yml ou .gitlab-ci.yml)
- Justification du choix GitHub Actions ou GitLab CI (une phrase minimum)
- Screenshot de l'onglet Actions/Pipelines montrant au moins une exécution réussie
- Screenshot détaillé de chaque étape de la pipeline (logs du lint, tests, build, deploy)
- Explication du fonctionnement de chaque étape de la pipeline (que fait-elle, pourquoi)
- Justification du choix du registry utilisé (GHCR, Docker Hub, GitLab Registry)
- Explication du tag par SHA de commit (pourquoi c'est préférable à un tag « latest »)
- Explication du health check : comment il fonctionne, pourquoi il fait échouer la pipeline

Exercice 7 – Gestion des secrets et déclencheurs (10 points)

a) Configurez les secrets nécessaires dans votre plateforme CI (tokens, credentials registry, etc.).

b) Configurez les déclencheurs : la pipeline doit se lancer automatiquement sur chaque push sur develop et sur chaque Pull/Merge Request vers main.

c) Expliquez par écrit : pourquoi ne faut-il jamais stocker de secrets en clair dans un fichier CI ? Quels risques concrets cela représente-t-il ?

📌 **Attendu dans le PDF :**

- Screenshot de la page de configuration des secrets (valeurs masquées, seuls les noms doivent être visibles)
- Liste des secrets configurés avec le rôle de chacun (ex : DOCKER_TOKEN → accès au registry)

- Extrait du fichier de pipeline montrant la section « on: » / « triggers » avec les déclencheurs
- Justification des déclencheurs choisis (pourquoi push sur develop ET PR vers main)
- Réponse rédigée sur les risques du stockage de secrets en clair (au moins 2 risques concrets)

Partie 4 – Orchestration et supervision (15 points)

Exercice 8 – Manifestes Kubernetes (10 points)

Sans nécessairement déployer sur un cluster, rédigez les manifestes YAML Kubernetes suivants :

a) Un Deployment pour le back-end avec 2 réplicas, l'image depuis votre registry, le port 3000 et une liveness probe sur /health.

b) Un Service de type ClusterIP pour exposer le back-end en interne.

c) Un Ingress (ou un Service NodePort/LoadBalancer) pour rendre le front-end accessible de l'extérieur.

d) Un Secret Kubernetes pour stocker le mot de passe de la base de données, référencé comme variable d'environnement dans le Deployment du back-end.

📌 **Attendu dans le PDF :**

- Contenu intégral de chaque fichier YAML (Deployment, Service, Ingress, Secret)
- Explication du rôle de chaque ressource Kubernetes (qu'est-ce qu'un Deployment, un Service, etc.)
- Justification du choix de 2 réplicas (pourquoi pas 1, pourquoi pas 3)
- Explication de la liveness probe : à quoi elle sert, que se passe-t-il si elle échoue
- Explication du choix entre Ingress et NodePort/LoadBalancer avec justification
- Explication de comment le Secret est injecté dans le pod (envFrom ou env.valueFrom)

Exercice 9 – Réflexion sur la supervision (5 points)

a) Expliquez comment vous mettriez en place une supervision en production. Quels outils utiliseriez-vous (Prometheus, Grafana, ELK, etc.) et quelles métriques surveilleriez-vous ?

b) Que se passe-t-il si un pod Kubernetes tombe ? Décrivez le mécanisme de self-healing.

📌 **Attendu dans le PDF :**

- Liste des outils de supervision choisis avec le rôle de chacun (collecte, stockage, visualisation, alerting)
- Liste des métriques surveillées (ex : CPU, mémoire, latence API, taux d'erreur 5xx…) avec justification
- Explication du self-healing Kubernetes : rôle du ReplicaSet, détection de panne, recréation du pod
- Justification du choix des outils (pourquoi Prometheus plutôt qu'un autre, etc.)

Partie 5 – Documentation (10 points)

Exercice 10 – README et documentation technique

Rédigez un fichier **README.md** à la racine de votre dépôt contenant :

- Une description du projet et de son architecture
- Les prérequis pour lancer le projet en local (outils, versions)

- Les commandes pour lancer l'application avec Docker Compose
- Une explication du fonctionnement de la pipeline CI/CD
- Les choix techniques et leurs justifications
- Un schéma d'architecture simplifié (Mermaid ou image)

📌 **Attendu dans le PDF :**

- Contenu intégral du README.md collé dans le PDF
- Schéma d'architecture intégré (screenshot du rendu Mermaid ou image)
- Vérification que le README est cohérent avec le reste de votre travail (mêmes outils, mêmes versions)
- Justification des choix techniques mentionnés dans le README

Récapitulatif des critères d'évaluation

Partie	Points	Durée indicative
Code fonctionnel et bien structuré	25 pts	
Pipeline CI/CD opérationnelle	30 pts	
Qualité des Dockerfiles et docker-compose	15 pts	
Pertinence des réponses de réflexion	10 pts	
README, captures d'écran et présentation du PDF	10 pts	
Manifestes Kubernetes corrects	**10 pts**	

— Fin du sujet E6 —